`python_cc/` — Confluent Cloud packaging wrapper for the scalar UDFs

> This directory is a **thin packaging wrapper** whose only job is to produce a Confluent Cloud–compatible sdist-in-ZIP artifact from the same PyFlink UDF source tree that the Confluent Platform path uses. It contains **no UDF source of its own**: the `scalar_udf/` package at `../python/src/scalar_udf/` is the single source of truth and is copied in at build time. The two reasons this exists as a separate project are (a) Confluent Cloud's Python UDF runtime pins `apache-flink==2.0.0` while the CP side needs `2.1.1` — one lockfile cannot satisfy both — and (b) CC requires the artifact to be shipped as a source distribution inside a ZIP, not as loose `.py` files.
>
> ⚠️ **Confluent Cloud Python UDFs are an Early Access feature.** Your CC organization must have the feature enabled for the artifact this directory produces to upload successfully.

Table of Contents

- **1.0 What's committed vs generated**
- **2.0 Building the CC artifact**
- **3.0 Why this is separate from `../python/`**
- **4.0 What ends up inside the ZIP**
- **5.0 See also**

1.0 What's committed vs generated

```
python_cc/
├── README.md              # (this file)
├── pyproject.toml         # COMMITTED — CC-specific project metadata
(scalar-udf-cc) pinned to apache-flink==2.0.0
├── .python-version        # COMMITTED — pins CPython 3.11 for the build
├── .gitignore             # COMMITTED — excludes src/, dist/, .venv/,
*.egg-info/
├── src/                   # GENERATED at build time — copied wholesale
from ../python/src/scalar_udf/
│   └── scalar_udf/        #   (same package, same code — the two language
paths share a source tree)
└── dist/                  # GENERATED at build time — uv build --sdist
output + the final ZIP
    ├── scalar_udf_cc-0.1.0.tar.gz
    └── scalar_udf_cc-0.1.0.zip   # ← what Terraform uploads as the
confluent_flink_artifact
```

The committed surface is deliberately tiny: **one pyproject.toml, one `.python-version`, one `.gitignore`**. Everything else is reproduced from the CP source tree on every build. This is the whole point — **no copy of the UDF `.py` files lives here** that could silently drift from `../python/src/scalar_udf/`.

2.0 Building the CC artifact

From the **project root** (where the top-level `Makefile` lives):

```
make build-scalar-udf-cc-python
```

The target performs four steps, all scoped to this directory:

Step	What it does
1	`rm -rf python_cc/src python_cc/dist` — fresh build tree
2	`cp -R ../python/src/scalar_udf python_cc/src/scalar_udf` — single-source-of-truth copy
3	`uv build --sdist` — produces `dist/scalar_udf_cc-0.1.0.tar.gz` via setuptools, honoring the constraint-deps block in `pyproject.toml`
4	`zip -FS -j dist/scalar_udf_cc-0.1.0.zip dist/scalar_udf_cc-0.1.0.tar.gz` — wraps the tarball in a ZIP, which is what CC's `confluent_flink_artifact` resource accepts as a `content_format = "ZIP"` + `runtime_language = "Python"` upload

The target does **not** invoke `terraform apply`. For the full deploy flow that consumes this ZIP see `../cc_deploy/README.md`.

3.0 Why this is separate from `../python/`

The CP path in `../python/` and the CC path here diverge on three concerns, each of which on its own would be a minor nuisance but together force a separate project:

Concern	CP path (`../python/`)	CC path (here)
`apache-flink` pin	`==2.1.1` (matches the `cp-flink` session-cluster image)	`==2.0.0` (CC's Python worker runtime, non-negotiable per CC docs)
Distribution format	Package installed into the baked venv via `uv pip install --no-deps .` inside the `cp-flink-python` Dockerfile	Source distribution inside a ZIP, uploaded via `confluent_flink_artifact`
Lockfile	`../python/uv.lock` resolves every dep incl. Presidio, boto3, etc. for local `uv run` + the cluster venv	No lockfile committed — the sdist declares its runtime dep on `apache-flink==2.0.0` and relies on CC's Python worker to provide it

A single `pyproject.toml` could technically loosen `apache-flink` to a version range covering both 2.0.0 and 2.1.1, but (a) Confluent's official flink-udf-python-examples repo explicitly instructs `apache-flink==2.0.0` for CC because the worker runtime is strict about it, and (b) even then the distribution format and lockfile stories would still diverge. Keeping `python_cc/` as a separate project makes the CC-specific concerns obvious and keeps `../python/` free to evolve its CP-specific dependency graph.

Project names are also deliberately distinct so the produced artifact filename is self-describing:

- `../python/pyproject.toml` → `name = "scalar-udf-python"` → used only inside the Dockerfile venv
- `pyproject.toml` → `name = "scalar-udf-cc"` → `scalar_udf_cc-0.1.0.tar.gz` / `.zip` (the filename that lands in `dist/`)

Both ship the **same `scalar_udf` package** under `src/`, so the `CREATE FUNCTION … AS 'scalar_udf.<module>.<symbol>'` address works identically on CP and CC. Only the project (distribution) name differs.

4.0 What's inside the ZIP

The final `dist/scalar_udf_cc-0.1.0.zip` is a one-entry ZIP wrapping the sdist tarball:

```
scalar_udf_cc-0.1.0.zip
└── scalar_udf_cc-0.1.0.tar.gz
    └── scalar_udf_cc-0.1.0/
        ├── PKG-INFO
        ├── pyproject.toml
        ├── setup.cfg
        └── src/
            └── scalar_udf/
                ├── __init__.py
                ├── celsius_to_fahrenheit.py
                ├── fahrenheit_to_celsius.py
                ├── pseudonymize_pii.py
                ├── resolve_identity.py
                ├── tokenize_pan.py
                ├── mask_pii_regex.py
                ├── dedup_key.py
                ├── consistent_bucket.py
                └── secrets_resolver.py
```

When CC's Python worker installs this sdist, it installs the `scalar_udf` package into the worker's site-packages. All eight UDF modules are physically present inside the ZIP regardless of which ones are actually registered via `CREATE FUNCTION` — today the terraform in `../cc_deploy/` only registers `celsius_to_fahrenheit` and `fahrenheit_to_celsius`, but extending that terraform to cover the other six is a pure-SQL change (no rebuild needed).

5.0 See also

- **`../python/README.md`** — tour of the eight UDFs and the CP deployment path (the authoritative source tree)
- **`../cc_deploy/README.md`** — Terraform deployment flow that consumes the ZIP built here
- **`../cp_deploy/README.md`** — the CP deployment path for the same UDFs (no ZIP involved; the package is baked into the `cp-flink-python` image)
- Confluent Cloud for Apache Flink — Create a User-Defined Function — upstream docs for the Python UDF Early Access feature

- confluentinc/flink-udf-python-examples — reference repo that the sdist-in-ZIP packaging pattern here is drawn from
- uv — fast Python package & project manager